Chapter 4: CRUD Operations with Forms and Requests

Project setup

Now, you will make dedicated pages for the CRUD operations. Create a file structure that looks like this:

```
your_project/
├── app.py
└── templates/
    ├── home.html
    ├── add_user.html
    ├── edit_user.html
    └── view_user.html
```

Install the required packages:

```
pip install flask flask-sqlalchemy pymysql
```

Form pages

Copy the following code snippets into their respective HTML files. They only introduce HTML tables and forms as the input and output interfaces.

templates/add_user.html

```
<!DOCTYPE html>
<html>
<head>
    <title>Add User</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
</head>
<body>
    <div class="container mt-5">
        <h1>Add New User</h1>
        {% with messages = get_flashed_messages(with_categories=true) %}
        {% if messages %}
        {% for category, message in messages %}
        <div class="alert alert-{{ category }}">{{ message }}</div>
        {% endfor %}
        {% endif %}
        {% endwith %}
        <form method="POST">
            <div class="mb-3">
                <label for="username" class="form-label">Username</label>
                <input type="text" class="form-control" id="username"
name="username" required>
            </div>
            <div class="mb-3">
                <label for="email" class="form-label">Email</label>
                <input type="email" class="form-control" id="email" name="email"
required>
            </div>
            <button type="submit" class="btn btn-primary">Add User</button>
            <a href="{{ url_for('index') }}" class="btn btn-secondary">Cancel</a>
        </form>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
</body>
</html>
```

- `<form method="POST">` creates a native HTML form element that sends all fields within it to the same URL (in this case `/add`) with the method `POST` instead of the regular `GET`.
- `<input>` elements contain the variables. The variable names are set by the `name` attribute.Add `type` and `required` attributes where necessary.

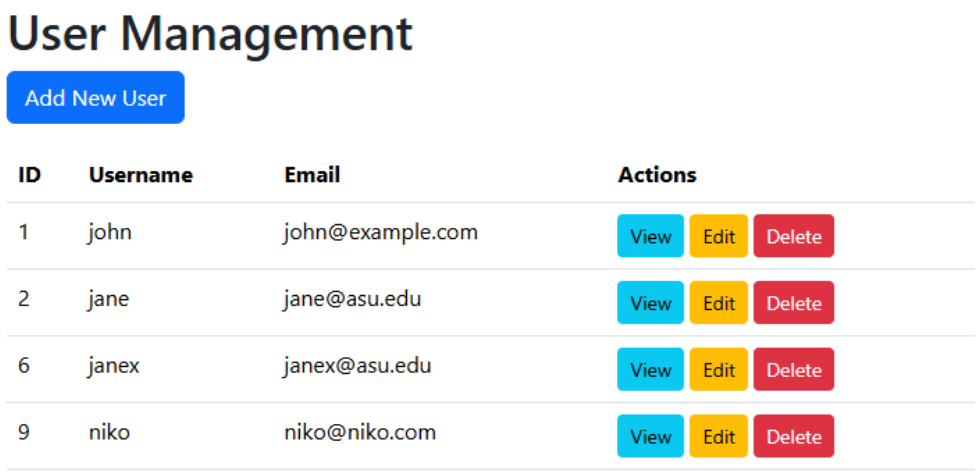
- `<button type="submit">` then makes the POST API request, transmitting the filled out form fields.

templates/home.html

```
<!DOCTYPE html>
<html>
<head>
    <title>User Management</title>
    <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
EVSTQN3/azprG1Anm3QDgpJLIm9Nao0Yz1ztcQTwFspd3yD65VohhpuuCOmLASjC"
crossorigin="anonymous">
</head>
```

templates/home.html (continued)

```
<body>
    <div class="container mt-5">
        <h1>User Management</h1>
        <!-- Flash messages -->
        {% with messages = get_flashed_messages(with_categories=true) %}
            {% if messages %}
                {% for category, message in messages %}
                <div class="alert alert-{{ category }}">{{ message }}</div>
                {% endfor %}
            {% endif %}
        {% endwith %}
        <a href="{{ url_for('add_user') }}" class="btn btn-primary mb-3">Add New
User</a>
        <table class="table">
            <thead>
                <tr>
                    <th>ID</th>
                    <th>Username</th>
                    <th>Email</th>
                    <th>Actions</th>
                </tr>
            </thead>
            <tbody>
                {% for user in users %}
                <tr>
                    <td>{{ user.id }}</td>
                    <td>{{ user.username }}</td>
                    <td>{{ user.email }}</td>
                    <td>
                        <a href="{{ url_for('view_user', id=user.id) }}"
class="btn btn-info btn-sm">View</a>
                        <a href="{{ url_for('update_user', id=user.id) }}"
class="btn btn-warning btn-sm">Edit</a>
                        <a href="{{ url_for('delete_user', id=user.id) }}"
class="btn btn-danger btn-sm"
                            onclick="return confirm('Are you sure?')">Delete</a>
                    </td>
                </tr>
                {% endfor %}
            </tbody>
        </table>
    </div>
    <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.0.2/dist/js/bootstrap.bundle.min.js"
integrity="sha384-
MrcW6ZMFYlzcLA8Nl+NtUVF0sA7MsXsP1UyJoMp4YLEuNSfAP+JcXn/tWtIaxVXM"
crossorigin="anonymous"></script>
</body>
</html>
```



Test your understanding of how to create HTML forms by making the missing one to view an individual user and the one to edit an individual user! Make sure they work by adding, editing, and deleting a couple of users.

Updates to the server code

You need to make changes to the server code of the previous chapter. Primarily, you will no longer pass information as URL parameters (e.g., `/add_user/<str:username>/`). Instead, you will use **POST requests** and transmit the new variables in the request body.

Routes will now look like this:

```python
@app.route('/add', methods=['GET', 'POST'])  ❶
def add_user():
    if request.method == 'POST':  ❷
        username = request.form['username']  ❸
        email = request.form['email']
        if not username or not email:  ❹
            flash('Please fill in all fields', 'error')
            return redirect(url_for('add_user'))
        try:
            new_user = User(username=username, email=email)
            db.session.add(new_user)
            db.session.commit()
            flash('User added successfully!', 'success')
            return redirect(url_for('index'))  ❺
        except Exception as e:
            flash(f'Error adding user: {str(e)}', 'error')
            return redirect(url_for('add_user'))  ❻

    return render_template('add_user.html')  ❼
```

❶ The route now accepts both "GET" and "POST" as a method.

❷ The database manipulation part of the function is only executed if the request method is "POST".

❸ This extracts what the user typed into the form field with the label "username".

❹ If one of the fields is missing, the server shows an error message and reloads the page to show it.

❺ If the new user has been added successfully, the server redirects the browser to the landing page.

❻ If there was an exception when adding the user, the server shows a different error message and reloads the page to show it.

❼ If the method is "GET", this function simply renders the template.

The sever is still accepting GET requests

- `GET` (e.g., just typing in the URL) will render and return the template.
- `POST` (i.e., form action) will make database changes.
- In real-world systems you would use more specific HTTP methods like `PATCH`, `PUT`, and `DELETE`. To keep it simple, stick to `POST` for this class.

Make sure you understand every line of the `add_user()` function and then insert it into the template for the server code below. Then implement the remaining three of the four CRUD

functions. Note that you have to import `request` from the Flask package for this.

app.py

```
from flask import Flask, render_template, redirect, url_for, flash, request
from flask_sqlalchemy import SQLAlchemy

app = Flask(__name__)

app.config['SQLALCHEMY_DATABASE_URI'] =
'mysql+pymysql://root:mysqlrootpassword@localhost:3306/flask_app'
app.config['SQLALCHEMY_TRACK_MODIFICATIONS'] = False
app.config['SECRET_KEY'] = 'your-secret-key-here'

db = SQLAlchemy(app)

class User(db.Model):
    id = db.Column(db.Integer, primary_key=True)
    username = db.Column(db.String(80), unique=True, nullable=False)
    email = db.Column(db.String(120), unique=True, nullable=False)

with app.app_context():
    db.create_all()

@app.route('/')
def index():
    users = User.query.all()
    return render_template('home.html', users=users)

# CREATE - Add new user
@app.route('/add', methods=['GET', 'POST'])
def add_user():
    # TODO: Implement add user functionality
    return render_template('add_user.html')

# READ - View an individual user
@app.route('/view', methods=['GET', 'POST'])
def view_user():
    # TODO: Implement view user functionality
    return render_template('view_user.html')

# UPDATE - Edit an individual user
@app.route('/update', methods=['GET', 'POST'])
def update_user():
    # TODO: Implement update user functionality
    return render_template('edit_user.html')

# DELETE - Delete a user
@app.route('/delete', methods=['POST'])
def delete_user():
    # TODO: Implement delete user functionality
    return redirect(url_for('index'))
```